RAG Assignment: Baseline vs. Re-Ranking

Name: Saksham **GitHub:** https://github.com/Ujjiyara/rag-lma-assignment

Setup

Item	Choice
Document	CATAN Game Rules & Almanac (5th ed., 16 pages)
Chunking	106 chunks, ~90-word target (median 96), sentence-based, 1-sentence overlap, no chunk spans two pages
Bi-Encoder	`all-MiniLM-L6-v2` (384-d, cosine similarity)
Cross-Encoder	`cross-encoder/ms-marco-MiniLM-L-6-v2`
LLM	`gemini-flash-lite-latest`, temperature 0, same prompt for both pipelines
Retrieval	Top-10 from the bi-encoder → Top-3 to the LLM

I picked the Catan rulebook because it states a rule once in the short Game Rules section and then adds exceptions and limits later in the Almanac. So the chunk that looks most similar to a question is often not the one with the actual answer.

For each failure query I marked a “gold chunk”, the chunk containing the sentence that decides the answer. It is found by a phrase match in code, so the ranks are not edited by hand.

1. Architecture Comparison

A **bi-encoder** encodes the query and each chunk separately into one vector each, and relevance is the cosine similarity between them. Chunk vectors are computed once at indexing time, so a query only needs one encoding and a dot product. The downside is that the whole chunk is squeezed into one vector before the model ever sees the question, so small but important words like “not”, “only”, “either…or” and “except” get lost. The vector mostly captures the topic.

A **cross-encoder** takes the query and chunk together as one input (`[CLS] query [SEP] chunk`). Self-attention lets every query token look at every chunk token, and the model outputs a single relevance score. This is why, for Q3, it scored the “either 1 knight card or 1 progress card” chunk far above the rest (9.50 vs. 5.24 for the next one). The cost is that nothing can be precomputed: every (query, chunk) pair needs a full forward pass at query time.

This showed up in my scores. For Q4 the cosine scores of all 10 candidates were bunched between 0.52 and 0.62, so the bi-encoder could barely separate them. The cross-encoder scores ranged from 7.84 for the gold chunk down to −2.70.

2. Qualitative & Rank Shift Analysis

Query	Type	Gold chunk	Bi-encoder rank	Cross-encoder rank
Q1: On the ore 2:1 harbor, can I trade wool at 3:1?	Keyword trap	57	#4	#3
Q2: Which hex never receives a number token?	Negation	83	#6	#1
Q3: Can I play a knight and a progress card in the same turn?	Constraint	39	#6	#1
Q4: I have built all 5 settlements. How can I build another one?	Corner case	76	#8	#1

Query	Answer 1 (baseline)	Answer 2 (re-ranked)
Q1	Says no rule covers it, points to the generic 3:1 harbor	No, “not even 3:1” ✔
Q2	Talks about there being no “7” token, hedges toward the desert, says the rule is missing	The desert ✔
Q3	“No rule explicitly states…”	Only 1 development card per turn, so no ✔
Q4	“You cannot build another one” ✘	Upgrade a settlement to a city first ✔

Q1 (keyword trap). The baseline’s #1 was chunk 58 (“…if he traded 4 wool instead of 4 ore…”), which shares the words ore, wool and trade but doesn’t answer the question. The real rule (“a special harbor does not permit… not even 3:1”) was at #4, just outside the cut-off. Answer 1 omitted the rule and was misleading. The cross-encoder also pushed chunk 58 down from #1 to #8.

Q2 (negation). The baseline ranked the general Number Tokens entry first. The answer is a one-line note inside the Variable Set-up section, surrounded by unrelated text, so its vector didn’t match the question well. Answer 1 partly guessed: it mixed up “no 7 token” with the question.

Q3 (constraint). The baseline top 3 described how knight and progress cards work, matching the words “knight” and “progress”. The limit is written using the general term “development card”, so it ranked #6. Answer 1 said the rulebook doesn’t cover it, which is an omission.

Q4 (corner case). The answer is a short note at the end of an example paragraph about the Distance Rule, so it was buried at #8 behind generic “how to build” chunks. Answer 1 stated a wrong rule (“you cannot build another one”). This is the worst failure, because it is confident and wrong.

In all four cases, Answer 1 failed because the deciding sentence was outside the top 3. The LLM was told to use only the excerpts, so it can only be as good as its context. Answer 2 was correct every time.

Wider check (29 rule questions, retrieval only). I picked Q1–Q4 from a larger pool as the cases where the gold chunk was at ranks 4–10. Over the whole pool:

Metric	Pipeline 1	Pipeline 2
Hit@1	0.345	0.655
Hit@3 (what the LLM sees)	0.655	0.828
Hit@10	0.862	0.862
MRR@10	0.533	0.741

11 improved, 13 unchanged, 1 got worse, 4 had the gold chunk outside the top 10.

Hit@10 is the same for both because re-ranking only reorders the same 10 candidates. It improves precision at the top but can't improve recall. One query got worse ("give resource cards for free", #4 → #6), probably because the rulebook says "give away cards" rather than "for free". The 4 misses (e.g. "Can I trade 2 wool for 1 wool?") can't be fixed by re-ranking at all. A larger k, hybrid BM25 + dense search, or a better embedding model would be needed there.

3. Latency Profiling

CPU only. Retrieval timed over 80 runs after warm-up; LLM times are from 4 calls per pipeline and include network delay.

Stage	Mean (ms)	Median (ms)
Pipeline 1 retrieval (embed + top-10)	14.6	13.9
Cross-encoder re-rank only (10 pairs)	168.6	136.2
Pipeline 2 retrieval (embed + top-10 + re-rank)	201.1	184.1
LLM generation (Answer 1 / Answer 2)	1196.7 / 1039.2	1085.7 / 1022.3

	Retrieval (ms)	LLM (ms)	Total (ms)
Pipeline 1	13.9	1085.7	1099.6
Pipeline 2	184.1	1022.3	1206.4

Re-ranking makes retrieval about 13× slower, but end to end it adds only about 10%, because the LLM call takes most of the time. For roughly 170 ms extra, it fixed all four failure queries, so I think it is worth it. The LLM numbers are rough (only 4 calls, network included), and the cross-encoder would be much faster on a GPU.

4. Scaling Question

Measured cost: 20.2 ms per (query, chunk) pair for the cross-encoder, 13.5 ms per chunk for bi-encoder embedding.

Corpus size	CE over all chunks (per query)	CE over Top-10 (per query)	Bi-encoder indexing (one-off)
106	2.1 s	0.20 s	1.4 s
10,000	~3.4 min	0.20 s	~2.2 min
1,000,000	~5.6 h	0.20 s	~3.7 h

(10k and 1M rows are linear extrapolations of the measured cost.)

A cross-encoder score depends on the query, so it can't be computed in advance. Running it over the whole collection means N forward passes for every query, and the cost grows linearly

with the corpus. At a million chunks that is hours per question. A bi-encoder moves the expensive part to indexing: chunk embeddings are computed once, and at query time we only encode the query and do fast vector search (which ANN indexes like FAISS make sub-linear at scale).

So the two stages split the work. The cheap bi-encoder is used for recall, cutting the collection down to 10 candidates, and the expensive cross-encoder is used for precision on just those 10, so its cost stays constant. The weakness is that stage 2 can only reorder what stage 1 found, as seen in my 4 queries that were never in the top 10.

Limitations

This is one document with 29 questions, and I chose the gold chunks myself. I used a Flash-Lite model because of free-tier limits, and small MiniLM models for both encoders. With more time I would try a larger k, a hybrid BM25 + dense first stage, and a stronger embedding model to fix the recall misses.